



How to Train a Model?

Applied Machine Learning

Building a ML Model

Assistant Professor | **Chun-Hsiang Chan**

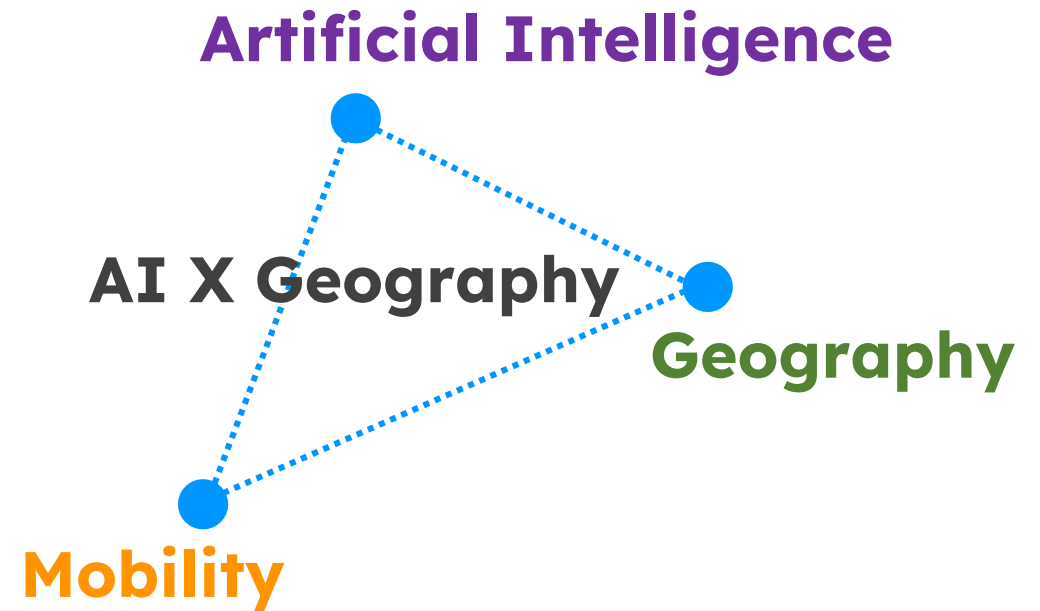
Department of Geography
National Taiwan Normal University



Outline

#GeoAI #Mobility #Geography #STDS

- 01 Training & Testing
- 02 Overfitting
- 03 Feature Engineering
- 04 Lab Practice
- 05 References



Training & Testing

Now we tell you how to train a model; first of all, we have to split your data into two parts:

Training dataset

To tell the model what does the data pattern look like?

Testing dataset

To test the model whether it could be generalizable?

Training & Testing

Think about a scenario ... how do you prepare a exam?



Fall 2026



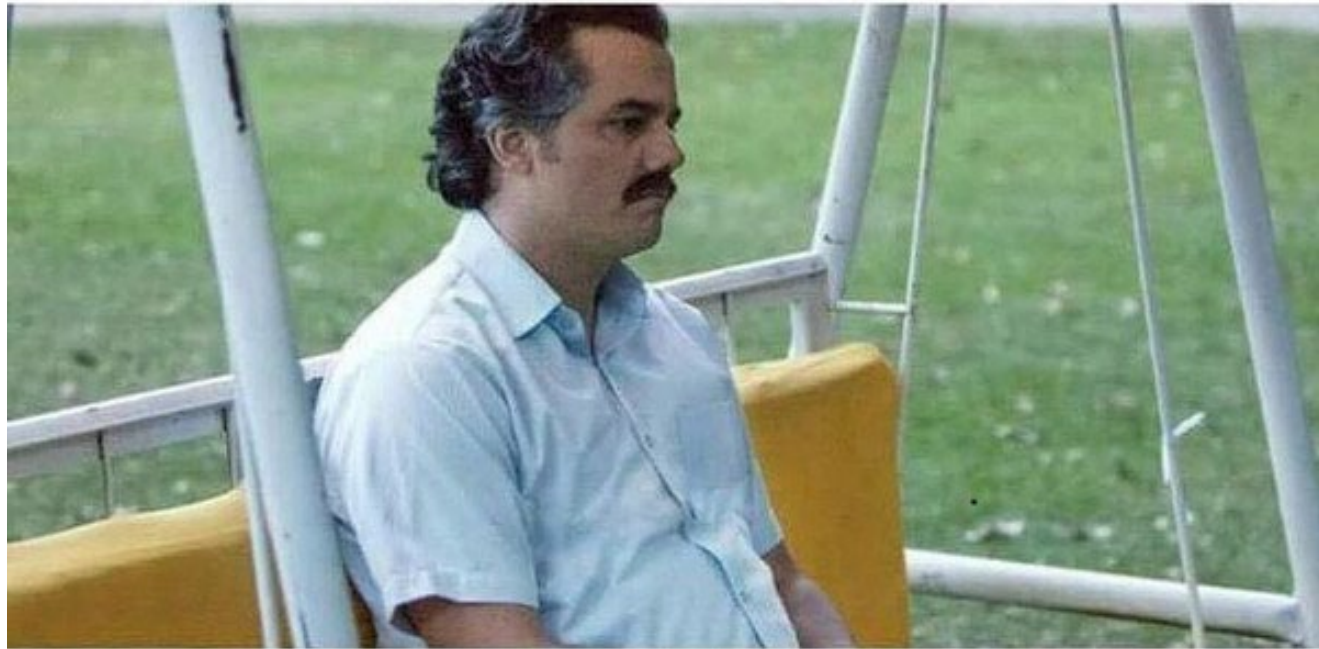
Chun-Hsiang Chan (2026)

Training & Testing

The **TRUTH** is ...

**Have you ever
think about
the reason
why?**

當你覺得自己已經算認真
但寫考古題有時候還是會不及格



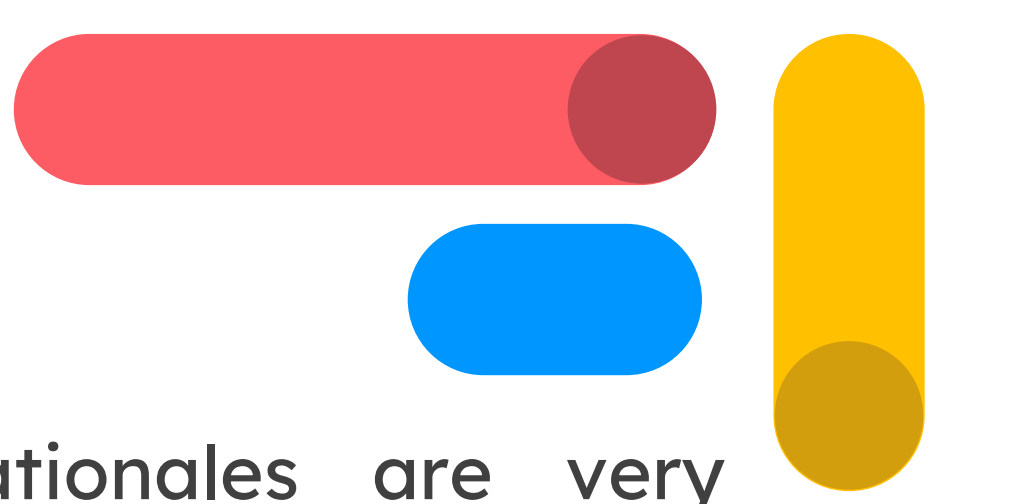
Training & Testing

The data patterns from your training dataset will dominate the learning directions of the model, and if the patterns between training and testing datasets are similar, and then your model will have a good model performance.

But it does not guarantee that your model could be generalized in other datasets.

Therefore, the representative of your training and testing data is the most important in the model training and evaluation process.

Training & Testing



Although the abovementioned rationales are very **straightforward**; however, it is lack of mathematical evidence to confirm that the hypotheses (rationales) is correct.

Later, we will prove it step by step from ...

- 1) Feasibility of learning
- 2) Theory of generalization
- 3) Interpreting the generalization bound
- 4) The learning curve

Training & Testing

First all of, let's think about “**Feasibility of learning!**”

The target function (f) is what the learning algorithm aims to discover, but it is unknown.

This leads to a fundamental question:

How can a limited set of observed data provide enough information to learn an unknown target function?

Training & Testing

Outside the Dataset

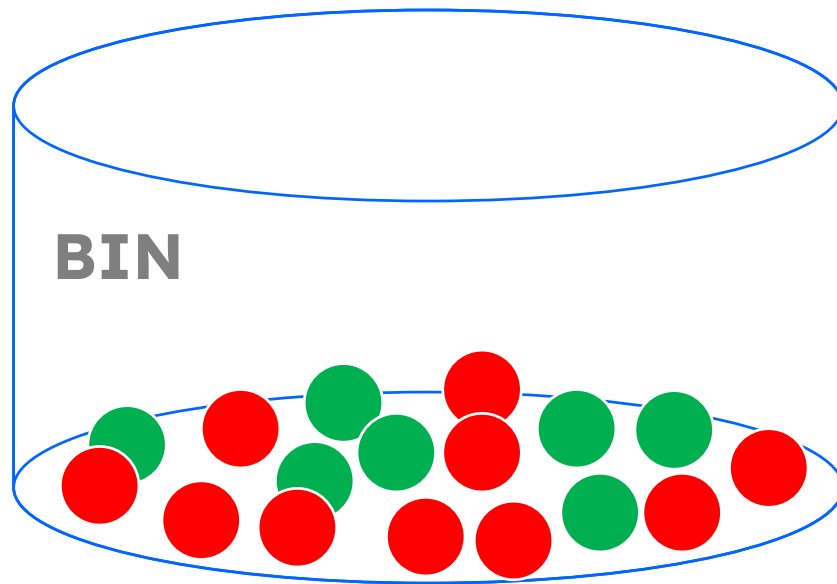
Once we obtain the training dataset $\mathcal{D}$, we know the value of the target function f for the examples contained in $\mathcal{D}$. However, this alone does not mean that we have learned f . Simply reproducing the known outputs for the training examples is only **memorization**.

Does the dataset $\mathcal{D}$ allow us to infer anything about f beyond the observed training examples?

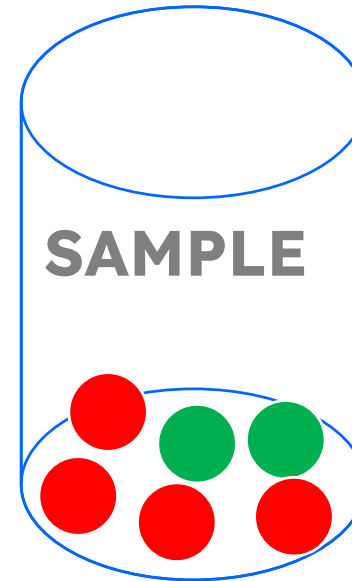
If the answer is yes, then genuine learning has occurred. If the answer is no, then the data provide no basis for generalization, and learning is not feasible.

Training & Testing

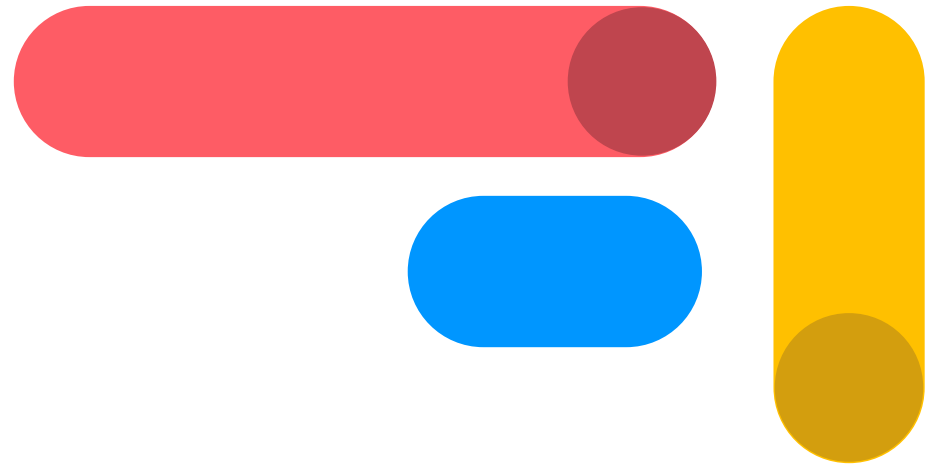
Let's take an example.



μ = probability of red marbles



ν = fraction of red marbles



Training & Testing

We randomly draw N marbles independently from the bin, with replacement, and record the fraction ν of red marbles in the sample.

At first, this may seem to tell us very little about the marbles we did not observe. For example, the sample could contain mostly green marbles even if the bin itself contains mostly red marbles.

The key question is therefore whether the observed sample proportion ν can still provide useful information about the true proportion of red marbles in the entire bin.

Training & Testing

The probability distribution of the random variable v in terms of the parameter μ is well understood, and when the sample size is big, v tends to be close to μ .

To quantify the relationship between v and μ , we use a simple bound called the **Hoeffding Inequality**. It states that for any sample size N ,

$$\mathbb{P}[|v - \mu| > \epsilon] \leq 2e^{-2\epsilon^2 N} \text{ for any } \epsilon > 0$$

Here, $\mathbb{P}[\cdot]$ denotes the probability of an event, in this case with respect to the random sample we pick, and ϵ is any positive we choose.

Training & Testing

We can imply the similar concept to **in-sample error**.

The error rate within the sample, which corresponds to v in the bin model, will be called **in-sample error**.

$$\begin{aligned} E_{in}(h) &= (\text{fraction of } \mathcal{D} \text{ where } f \text{ and } h \text{ disagree}) \\ &= \frac{1}{N} \sum_{n=1}^N [h(x_n) \neq f(x_n)] \end{aligned}$$

Where $[statement] = 1$ if the statement is true, and $= 0$ if the statement is false.

Training & Testing

We have made explicit the dependency of E_{in} on the particular h that we are considering. In the same way, we define the out-of-sample error.

$$E_{out}(h) = \mathbb{P}[h(x) \neq f(x)]$$

which corresponds to μ in the model.

We could use the same concept with Hoeffding Inequality,

$$\mathbb{P}[|E_{in}(h) - E_{out}(h)| > \epsilon] \leq 2e^{-2\epsilon^2 N}, \text{ for any } \epsilon > 0$$

Training & Testing

The way to get around this is to try to bound $\mathbb{P}[|E_{in}(g) - E_{out}(g)| > \epsilon]$ in a way that does not depend on which g the learning algorithm picks.

Since g has to be one of the h_m 's regardless of the algorithm and the sample, it is always true that

$$|E_{in}(g) - E_{out}(g)| > \epsilon \Rightarrow |E_{in}(h_1) - E_{out}(h_1)| > \epsilon$$

$$\text{Or } |E_{in}(h_2) - E_{out}(h_2)| > \epsilon$$

$$\text{Or } |E_{in}(h_3) - E_{out}(h_3)| > \epsilon$$

...

$$\text{Or } |E_{in}(h_M) - E_{out}(h_M)| > \epsilon$$

Training & Testing

We may simplify the equation as ...

$$\text{if } B_1 \Rightarrow B_2, \text{ then } \mathbb{P}[B_1] \leq \mathbb{P}[B_2]$$

And if $B_1, B_2, \dots, B_M$ are any events, then

$$\mathbb{P}[B_1 \text{ or } B_2 \text{ or } \dots \text{ or } B_M] \leq \mathbb{P}[B_1] + \mathbb{P}[B_2] + \dots + \mathbb{P}[B_M]$$

The second rule is known as the union bound.

$$\mathbb{P}[|E_{in}(g) - E_{out}(g)| > \epsilon] \leq \mathbb{P}[|E_{in}(h_1) - E_{out}(h_1)| > \epsilon]$$

$$\text{or } |E_{in}(h_2) - E_{out}(h_2)| > \epsilon \text{ or } \dots$$

$$\text{or } |E_{in}(h_M) - E_{out}(h_M)| > \epsilon] \leq \sum_{m=1}^M \mathbb{P}[|E_{in}(h_m) - E_{out}(h_m)| > \epsilon]$$

$$\mathbb{P}[|E_{in}(g) - E_{out}(g)| > \epsilon] \leq 2Me^{-2\epsilon^2 N}$$

Training & Testing

The complexity of $\mathcal{H}$

If the number of hypotheses M goes up, we run more risk that $E_{in}(g)$ will be a poor estimator of $E_{out}(g)$ according to ...

$$\mathbb{P}[|E_{in}(g) - E_{out}(g)| > \epsilon] \leq 2Me^{-2\epsilon^2 N}$$

M can be thought of as a measure of the ‘complexity’ of the hypothesis set $\mathcal{H}$ that we use.

Training & Testing

The complexity of f

A complex target function f should be harder to learn than a simple f .

$$\mathbb{P}[|E_{in}(g) - E_{out}(g)| > \epsilon] \leq 2Me^{-2\epsilon^2 N}$$

Do you the reason?

Training & Testing

Theory of Generalization

The **out-of-sample error** E_{out} measures how well our training on $\mathcal{D}$ has generalized to data that we have not seen before. E_{out} is based on the performance over the entire input space $\mathcal{X}$.

If we want to estimate the value of E_{out} using a sample of data points, these points must be ‘fresh’ test points that have not been used for training, similar to the questions on the final exam that have not been used for practice.

Training & Testing

Generalization Error

We can define generalization error as the discrepancy between E_{in} and E_{out} as

$$\mathbb{P}[|E_{in}(g) - E_{out}(g)| > \epsilon] \leq 2Me^{-2\epsilon^2 N}$$

Pick a tolerance level of δ , for example $\delta = 0.05$, and assert with probability at least $1 - \delta$ that

$$E_{out}(g) \leq E_{in}(g) + \sqrt{\frac{1}{2N} \ln \frac{2M}{\delta}}$$

Training & Testing

Effective Number of Hypothesis

Growth function, the quantity that will formalize the effective number of hypothesis. The growth function is what is what will replace M in the generalization bound.

Let $x_1, x_2, \dots, x_N \in \mathcal{X}$. The dichotomies generated by $\mathcal{H}$ on these points are defined by

$$\mathcal{H}(x_1, \dots, x_N) = \{h(x_1), \dots, h(x_N) | h \in \mathcal{H}\}$$

Training & Testing

The growth function is defined for a hypothesis set $\mathcal{H}$ by

$$m_{\mathcal{H}}(N) = \max_{x_1, \dots, x_N \in \mathcal{X}} |\mathcal{H}(x_1, \dots, x_N)|$$

Since $\mathcal{H}(x_1, \dots, x_N) \subseteq \{-1, +1\}^N$, the set of all possible dichotomies on any N points and the value of $m_{\mathcal{H}}(N)$ is at most $|\{-1, +1\}^N|$, hence,

$$m_{\mathcal{H}}(N) \leq 2^N$$

Training & Testing

Bounding the Growth Function

If we want to bound the growth function $m_{\mathcal{H}}(N)$ for all values of N by a simple polynomial based on this break point.

However, the fact is that

$$\sqrt{\frac{1}{2N} \ln \frac{2M}{\delta}} \rightarrow 0$$

Therefore, N should be ∞ , and this means that $m_{\mathcal{H}}(N) \rightarrow \infty$, since $m_{\mathcal{H}}(N) \leq 2^N$.

Training & Testing

Shattering

A hypothesis set $\mathcal{H}$ **shatters** N points if it can correctly realize **every possible labeling** of those points.

2^N possible 2^N possible labelings

Training & Testing

Vapnik-Chervonenkis (VC) Dimension

The VC dimension is the **largest number of points that can be shattered** by the hypothesis set.

$$d_{VC} = \max\{N: H \text{ can shatter } N \text{ points}\}; m_{\mathcal{H}}(N) \leq \sum_{i=1}^{d_{VC}} \binom{N}{i}; m_{\mathcal{H}}(N) \leq N^{d_{VC}} + 1$$

Therefore, the higher VC dimension, the greater model flexibility. However, greater flexibility can fit more patterns, but also increases the risk of overfitting.

The VC generalization bound, for any tolerance $\delta > 0$,

$$E_{out}(g) \leq E_{in}(g) + \sqrt{\frac{8}{N} \ln \frac{4m_{\mathcal{H}}(2N)}{\delta}}$$

Training & Testing

Sample Complexity

The sample complexity denotes how many training examples N are needed to achieve a certain generalization performance.

The performance is specified by two parameters, ϵ and δ .

How fast N grows as ϵ and δ become smaller indicates how much data is needed to get good generalization.

$$N \geq \frac{8}{\epsilon^2} \left(\frac{4m_{\mathcal{H}}(2N)}{\delta} \right) \Rightarrow N \geq \frac{8}{\epsilon^2} \left(\frac{4((2N)^{d_{VC}} + 1)}{\delta} \right)$$

Training & Testing

Penalty for Model Complexity

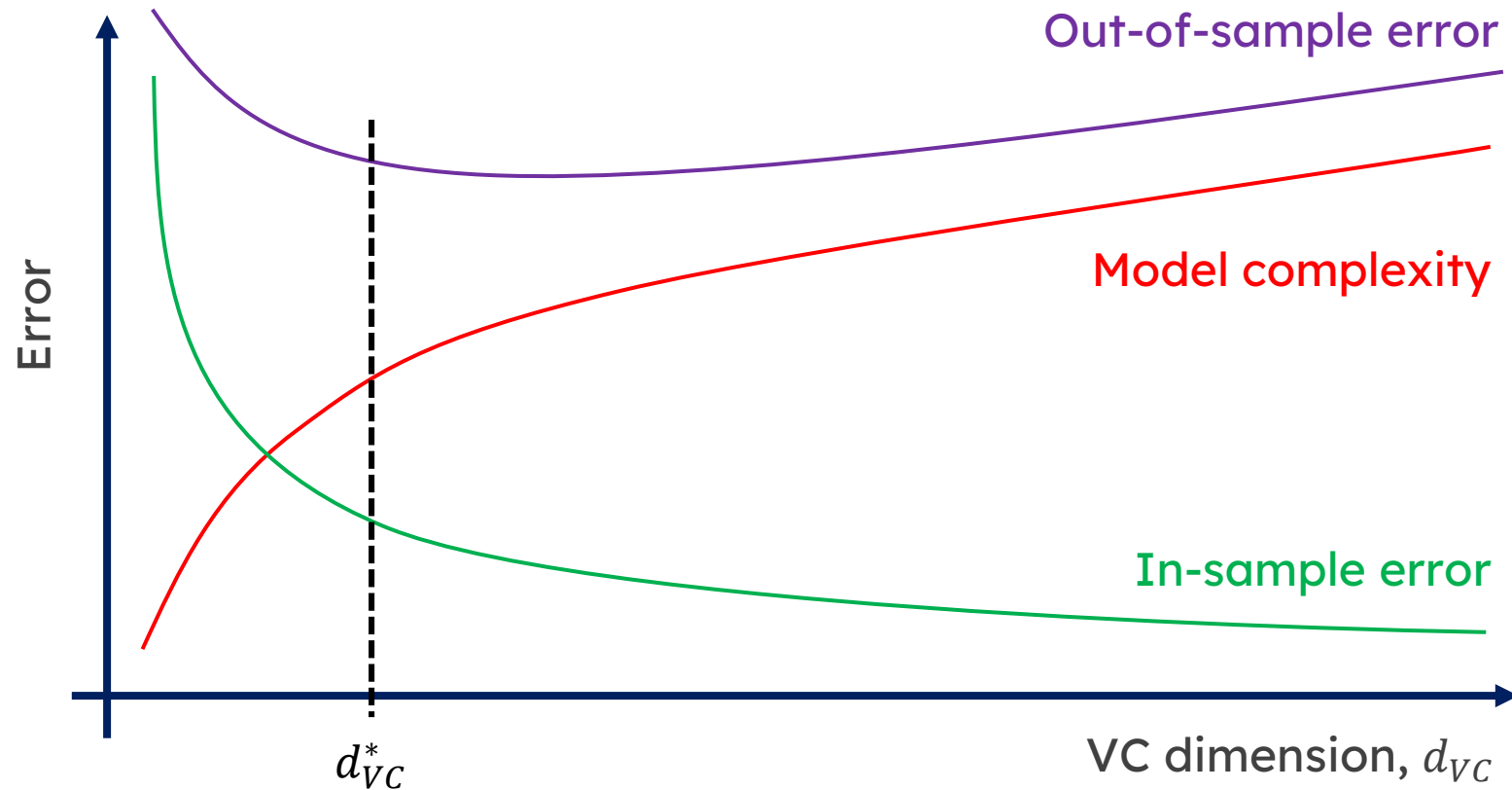
Sample complexity fixes the performance parameters ϵ (generalization error) and δ (confidence parameter) and estimates how many examples N are needed. In most practical situations, we are given a fixed dataset $\mathcal{D}$, so N is also fixed.

$$E_{out}(g) \leq E_{in}(g) + \sqrt{\frac{8}{N} \ln \frac{4m_{\mathcal{H}}(2N)}{\delta}}$$

$$E_{out}(g) \leq E_{in}(g) + \sqrt{\frac{8}{N} \ln \frac{4((2N)^{d_{VC}} + 1)}{\delta}}$$

Training & Testing

Let's see this concept via a figure.



Training & Testing

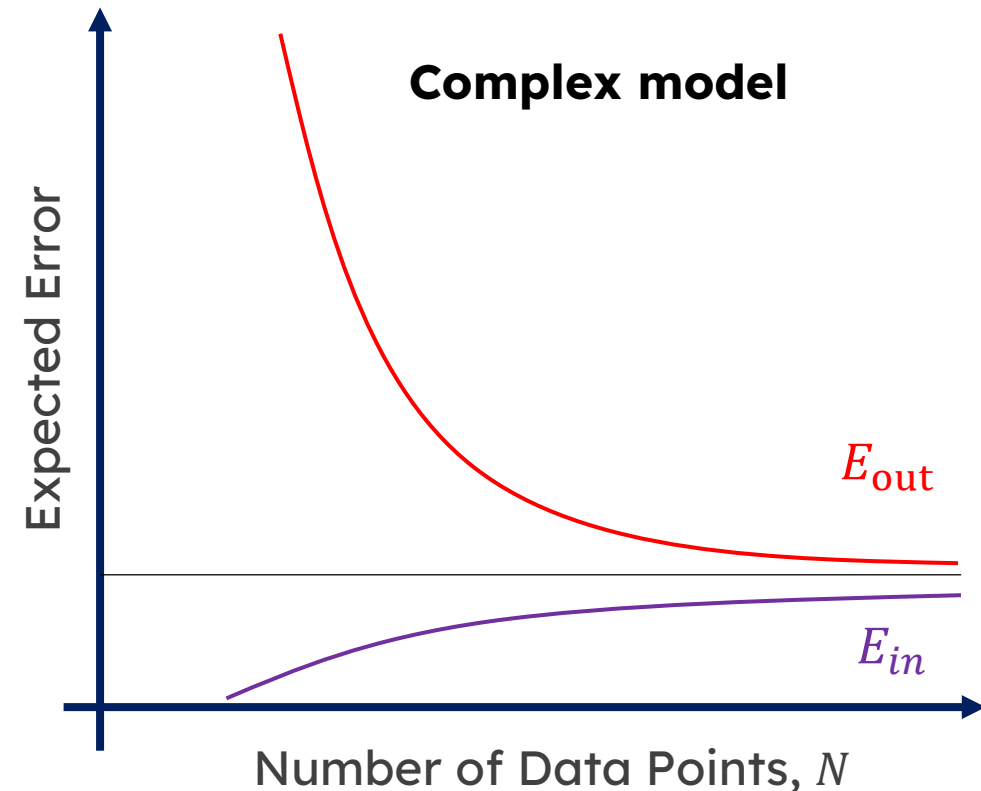
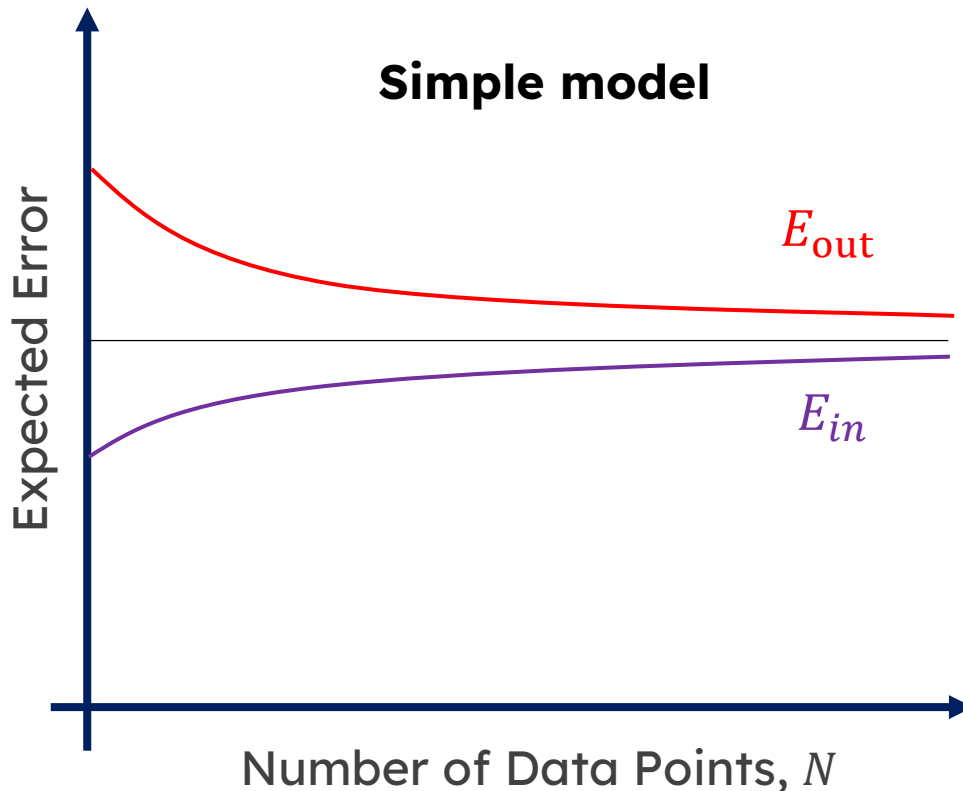
The Learning Curve

The learning curves summarize the behavior of the in-sample and out-of-sample errors as we vary the size of the training set.

After learning with a particular dataset $\mathcal{D}$ of size N , the final hypothesis $g^{(\mathcal{D})}$ has in-sample error $E_{in}(g^{(\mathcal{D})})$ and out-of-sample error $E_{out}(g^{(\mathcal{D})})$.

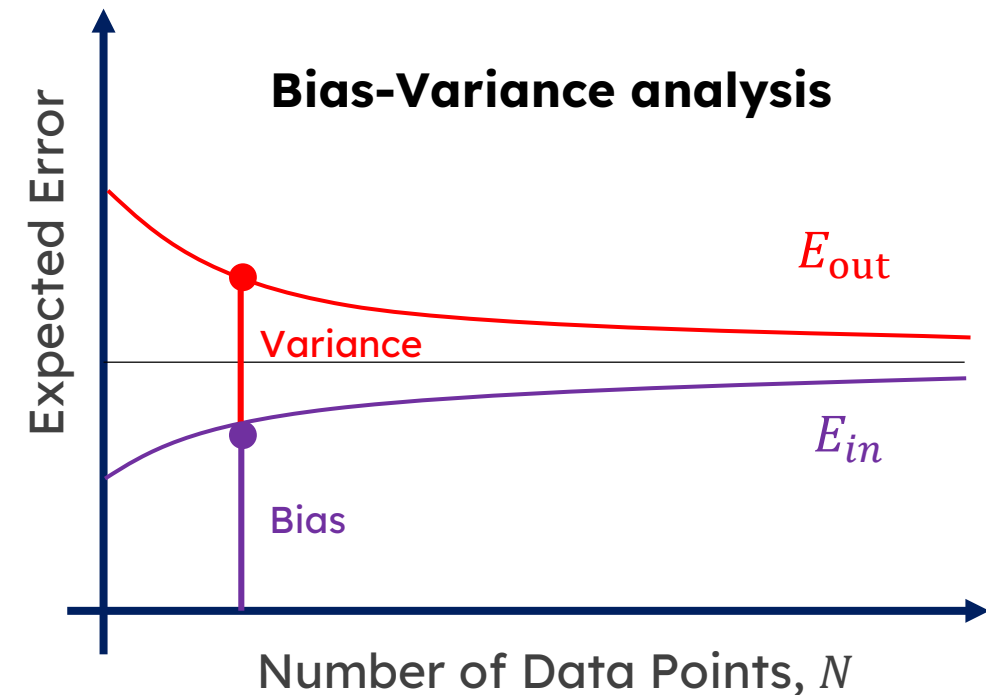
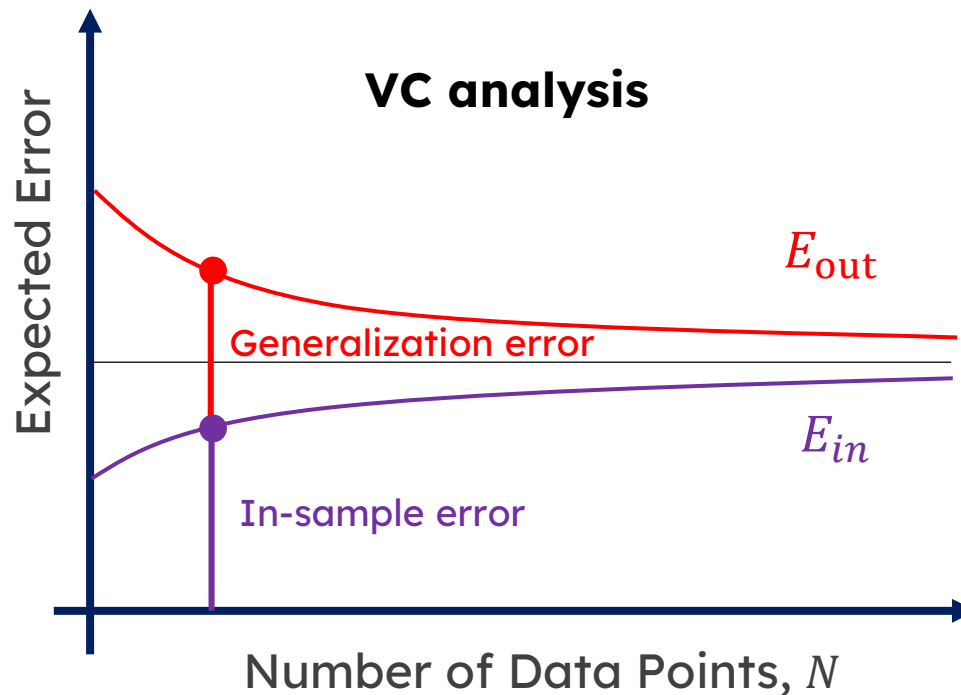
Training & Testing

If we visualize this concept, then it will be ...



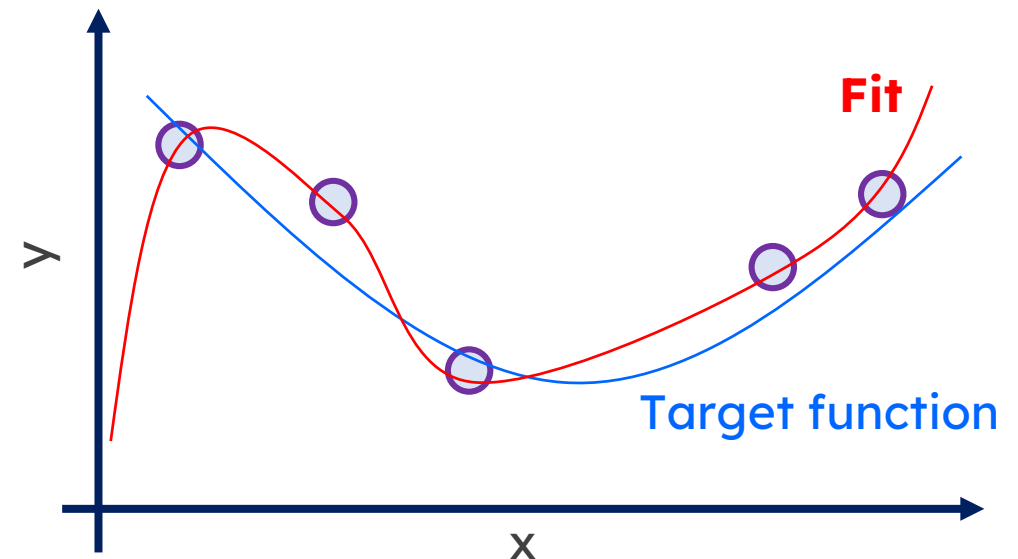
Training & Testing

In the VC analysis, E_{out} was expressed as the sum of E_{in} and a generalization error that was bound by Ω , the penalty for model complexity.

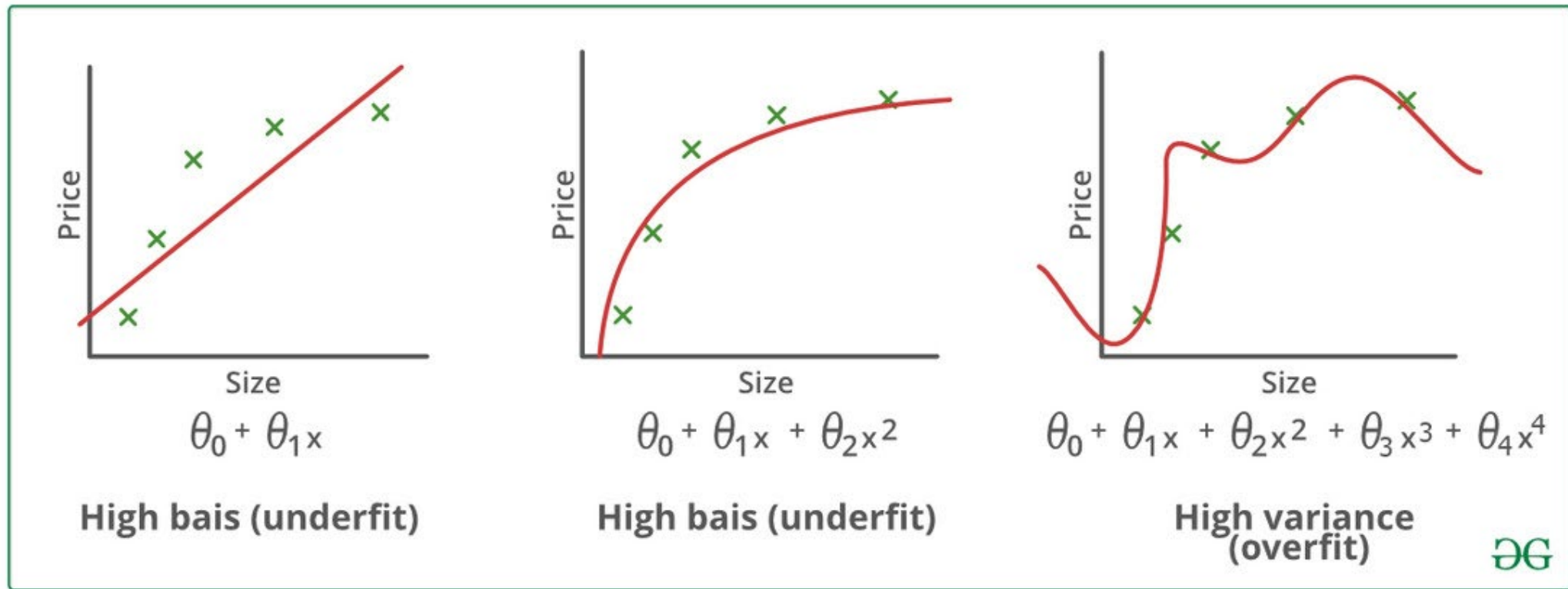


Overfitting

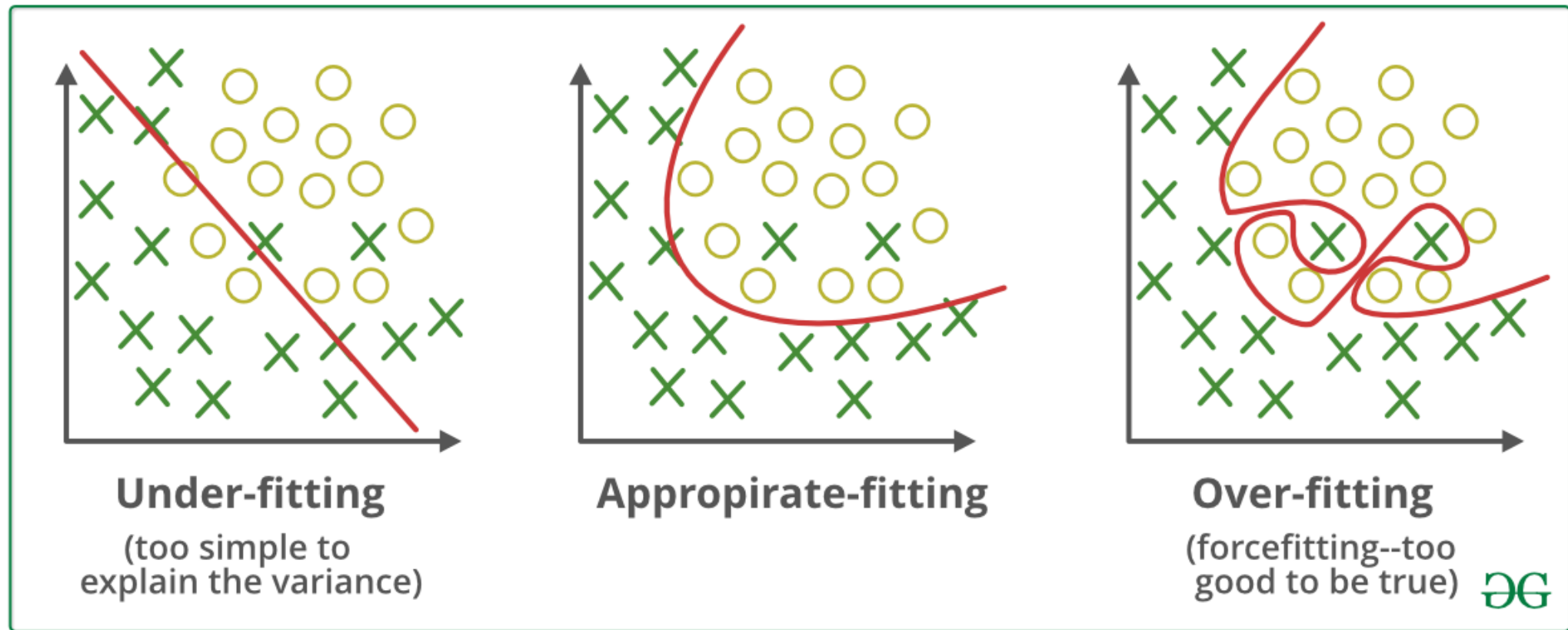
Overfitting literally means “fitting the data more than is warranted.” The main case of overfitting is when you pick the hypothesis with lower E_{in} , and it results in higher E_{out} . This means that E_{in} alone is no longer a good guide for learning.



Overfitting



Overfitting



Overfitting

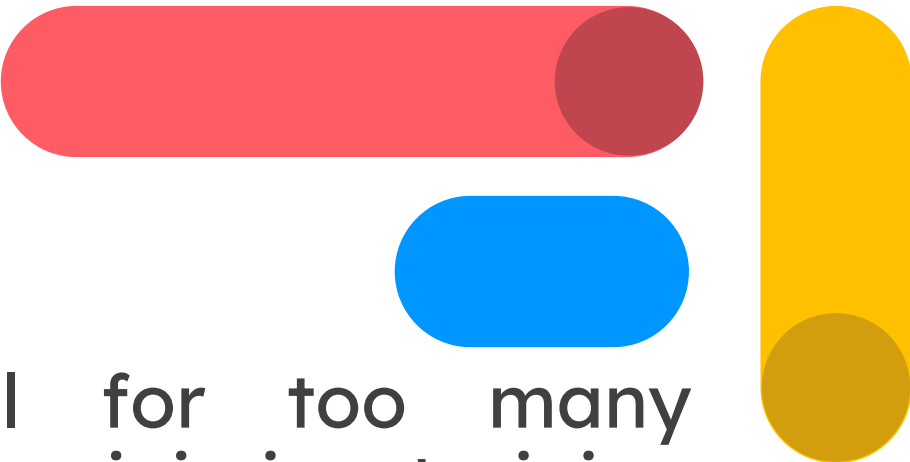
Reason of Overfitting

Excessive Model Complexity: The model has too many parameters relative to the problem (e.g., a high-degree polynomial function or a deep neural network without regularization), allowing it to memorize data points.

Insufficient Training Data: Small datasets don't provide enough examples to learn generalized patterns, making it easy for the model to fit every single data point perfectly.

Noisy Training Data: The dataset contains errors, outliers, or unrepresentative noise that the model mistakenly treats as valid signals.

Overfitting



Over-Training: Training a model for too many iterations allows it to continuously minimize training error at the expense of generalization.

Data Leakage: Information from the test or validation set accidentally leaks into the training dataset during preprocessing or feature engineering.

Feature Excess (High Dimensionality): Including too many irrelevant or redundant features gives the model unnecessary paths to fit random fluctuations in the dataset

Overfitting

How to solve overfitting?

Regularization

Add a penalty in objective function, such as Ridge Regression

$$\min_w \sum_i (y_i - w^T x_i)^2$$
$$\min_w \left[\sum_i (y_i - w^T x_i)^2 + \lambda \sum_j w_j^2 \right] \Rightarrow \text{Loss} + \lambda \|w\|_2^2$$

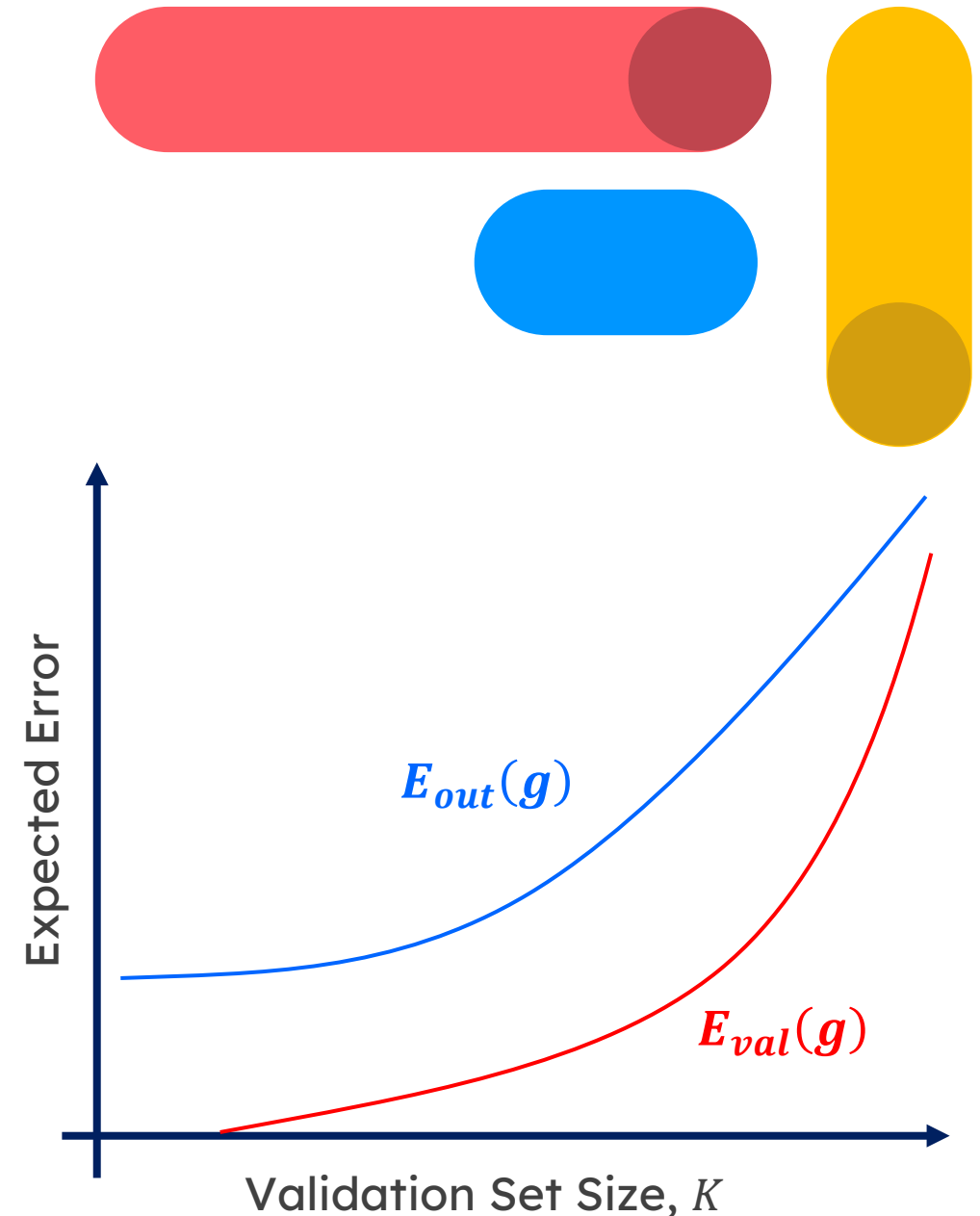
Lasso Regression

$$\min_w \left[E_{in}(w) + \lambda \sum_j |w_j| \right] \Rightarrow E_{in}(w) + \lambda \|w\|_1$$

Overfitting

Validation Set

The idea of a validation set is almost identical to that of a test set. We remove a subset from the data; this subset is not used in training. We then use this held-out subset to estimate the out-of-sample error. The held-out set is effectively out-of-sample, because it has not been used during the learning.



Overfitting

Cross Validation

Validation relies on the following chain of reasoning

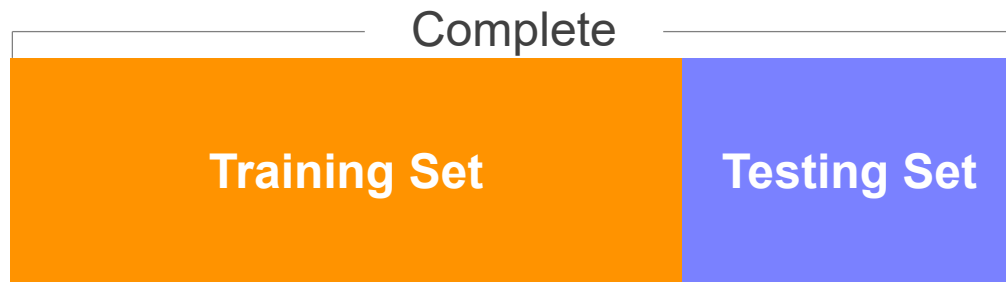
$$E_{out}(g) \approx E_{out}(g^-) \approx E_{val}(g^-)$$

which highlights the dilemma we face in trying to select K . We are going to output g . When K is large, there is a discrepancy between two out-of-sample errors $E_{out}(g^-)$ (which E_{val} directly estimates) and $E_{out}(g)$ (which is the final error when we learn using all the data $\mathcal{D}$).

We would like to choose K as small as possible to minimize the discrepancy between $E_{out}(g^-)$ and $E_{out}(g)$.

Overfitting

Cross Validation

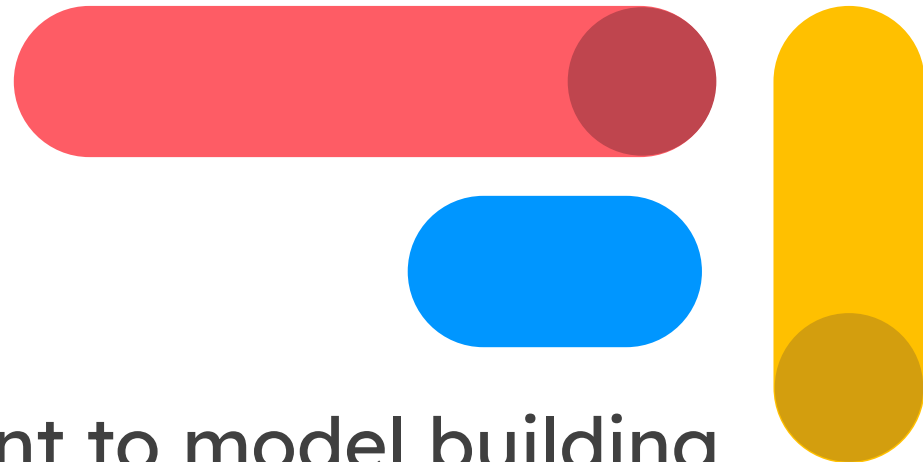


Cross Validation

To insure the model performance,
Training (70%): model training
Testing (30%): external testing

Furthermore, cross validation could be adopted for better model robustness, here, we demonstrate a 10-fold cross validation.

Feature Engineering



Feature engineering is also important to model building process, which will directly affect the patterns of training dataset and so does the learning curves of model training.

If you want to predict the housing rate, then you might collect several potential features, such as, city, town, latitude, longitude, transaction date, building age, floor, building type, income, and MRT (surrounding?), ...

Feature Engineering

Transformation

Sometimes, our data distribution follows a highly skewed shape; therefore, if we directly put this feature into model, then the model probably does not learn from this feature since the skewed value range.

To solve this issue, we usually adopt some mathematical transformation, such as

$$x = \text{income} \Rightarrow x' = \log(x) \text{ or } \sqrt{x}$$

Feature Engineering

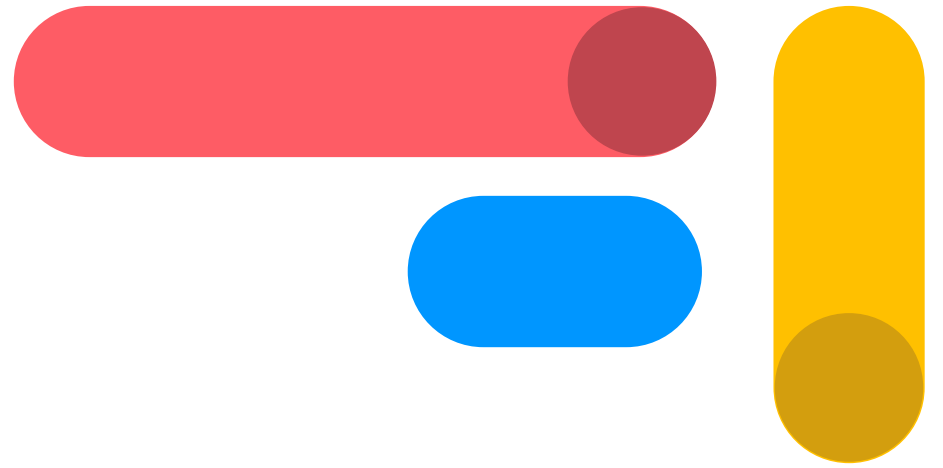
Encoding

In most geographic data, we have location label (categorical data) of each records; however, we cannot directly put the string data into the model training process. Hence, we here introduce a technique, **one-hot encoding**.

We may expand the number of categories as an array and assign 1 to the specific category and the others are 0.

$$x = \text{taipei}; \text{City} = \{\text{taipei}, \text{tokyo}, \text{london}\}$$
$$\text{taipei} = (1,0,0); \text{tokyo} = (0,1,0); \text{london} = (0,0,1)$$

Feature Engineering



Scaling

Typically, we have five numerical transformation methods.

1. Normalization
2. Standardization
3. Binarization
4. Centralization
5. Feature scaling

Among these five methods, standardization is the most common method in statistical analysis because of its range of transformed value.

Feature Engineering

Scaling

Methods	Equations
Standardization	$x' = \frac{x - \mu}{\sigma}$
Binarization	$\begin{aligned} &\text{if } x \geq \text{threshold}; x' = 1 \\ &\text{else } x' = 0 \end{aligned}$
Centralization	$x' = x - \frac{1}{n} \sum_{i=1}^n x_i$
Feature scaling	$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$

μ and σ denote mean and standard deviation, respectively.

Feature Engineering

Construction

In some cases, our features may be generated one or several new features, which are much relevant to the target function.

$$\begin{aligned} \text{speed} &= \frac{\text{distance}}{\text{travel time}}; \text{floor ratio} = \frac{\text{current floor}}{\text{total floors}}; \\ \text{population density} &= \frac{\text{population}}{\text{area}} \end{aligned}$$

Feature Engineering

Selection

To capture the nature of target function, we have to collect features as possible as we can; in addition, we may construct new features via original features. However, some original features might not be related to the target function; therefore, we should eliminate them.

So, **irrelevant variables**, **duplicated information**, **highly correlated variables**, and **noisy variables** should be solved before training the model.

Lab Practice

```
from sklearn.datasets import load_breast_cancer  
data = load_breast_cancer(as_frame=True)
```

```
X = data.data
```

```
y = data.target
```

```
print(X.shape)
```

```
print(y.value_counts())
```

```
print(data.target_names)
```

```
(569, 30)
```

```
target
```

```
1      357
```

```
0      212
```

```
Name: count, dtype: int64
```

```
['malignant' 'benign']
```

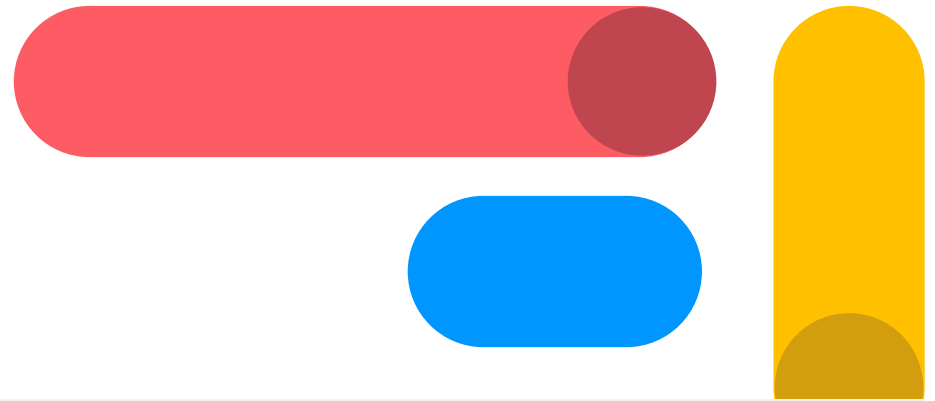
Lab Practice

#The data preview of X

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	0.2419	0.07871	...	25.380	17.33	184.60	2019.0	0.16220
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	0.1812	0.05667	...	24.990	23.41	158.80	1956.0	0.12380
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	0.2069	0.05999	...	23.570	25.53	152.50	1709.0	0.14440
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	0.2597	0.09744	...	14.910	26.50	98.87	567.7	0.20980
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	0.1809	0.05883	...	22.540	16.67	152.20	1575.0	0.13740
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.1726	0.05623	...	25.450	26.40	166.10	2027.0	0.14100
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.1752	0.05533	...	23.690	38.25	155.00	1731.0	0.11660
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.1590	0.05648	...	18.980	34.12	126.70	1124.0	0.11390
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.2397	0.07016	...	25.740	39.42	184.60	1821.0	0.16500
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.1587	0.05884	...	9.456	30.37	59.16	268.6	0.08996

569 rows × 30 columns

Lab Practice



X.columns

```
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',  
'mean smoothness', 'mean compactness', 'mean concavity', 'mean  
concave points', 'mean symmetry', 'mean fractal dimension', 'radius  
error', 'texture error', 'perimeter error', 'area error', 'smoothness error',  
'compactness error', 'concavity error', 'concave points error', 'symmetry  
error', 'fractal dimension error', 'worst radius', 'worst texture', 'worst  
perimeter', 'worst area', 'worst smoothness', 'worst compactness', 'worst  
concavity', 'worst concave points', 'worst symmetry', 'worst fractal  
dimension'], dtype='object')
```

Lab Practice

```
import numpy as np
from sklearn.model_selection import train_test_split
# split training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, stratify=y, random_state=42)
# print the sizes of training and testing datasets
print("Training:", X_train.shape)
print("Testing:", X_test.shape)
```

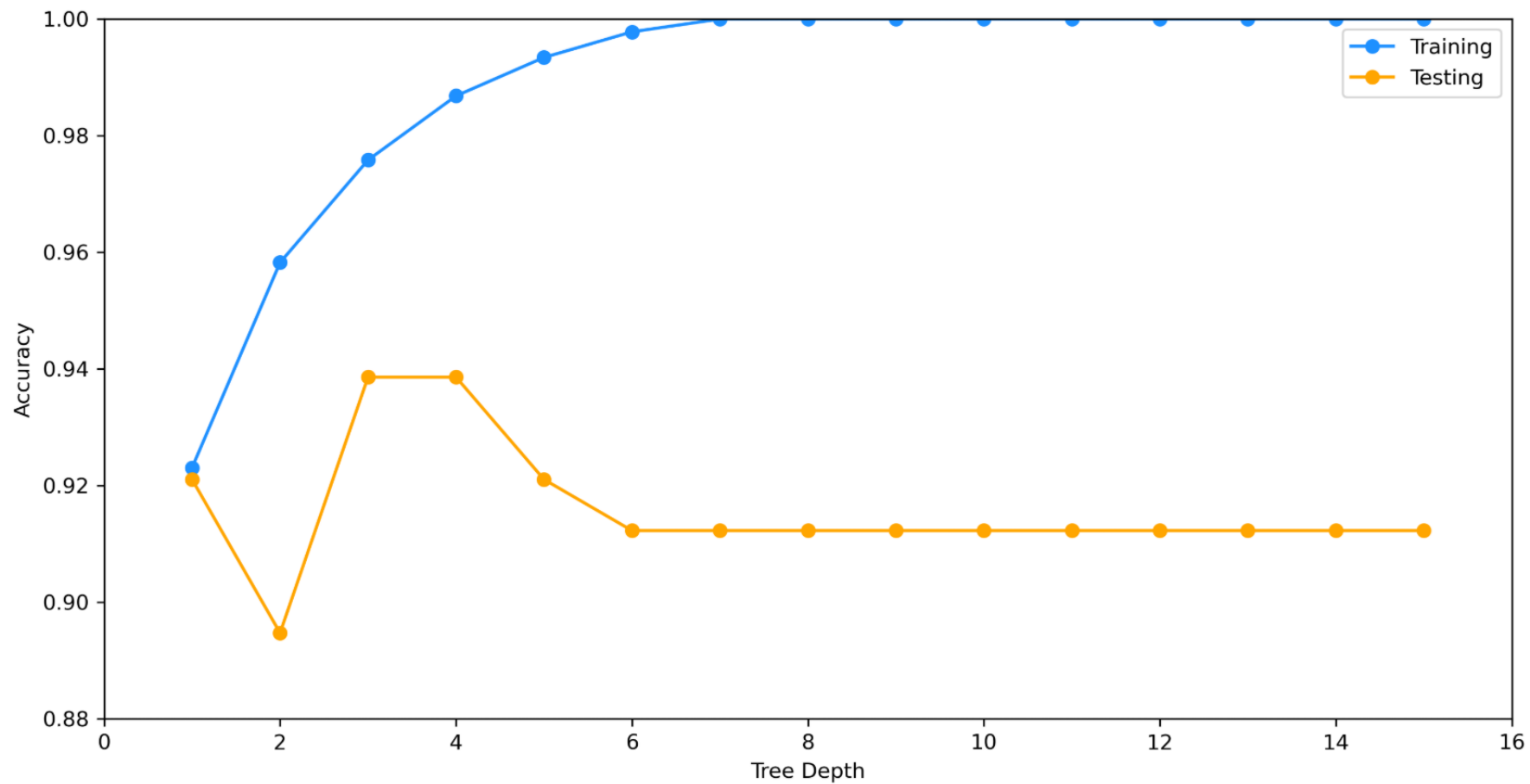
Lab Practice

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
# set hyperparameters
depths = range(1, 16)
train_acc = []
test_acc = []
for depth in depths:
    model=DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    train_acc.append(accuracy_score(y_train, model.predict(X_train)))
    test_acc.append(accuracy_score(y_test, model.predict(X_test)))
```

Lab Practice

```
import matplotlib.pyplot as plt
plt.figure(figsize=(12,6),dpi=300)
plt.plot(depths,train_acc,marker="o",c='dodgerblue',label="Training")
plt.plot(depths, test_acc, marker="o", c='orange', label="Testing")
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.axis([0, 16, 0.88, 1])
plt.legend()
plt.show()
```

Lab Practice



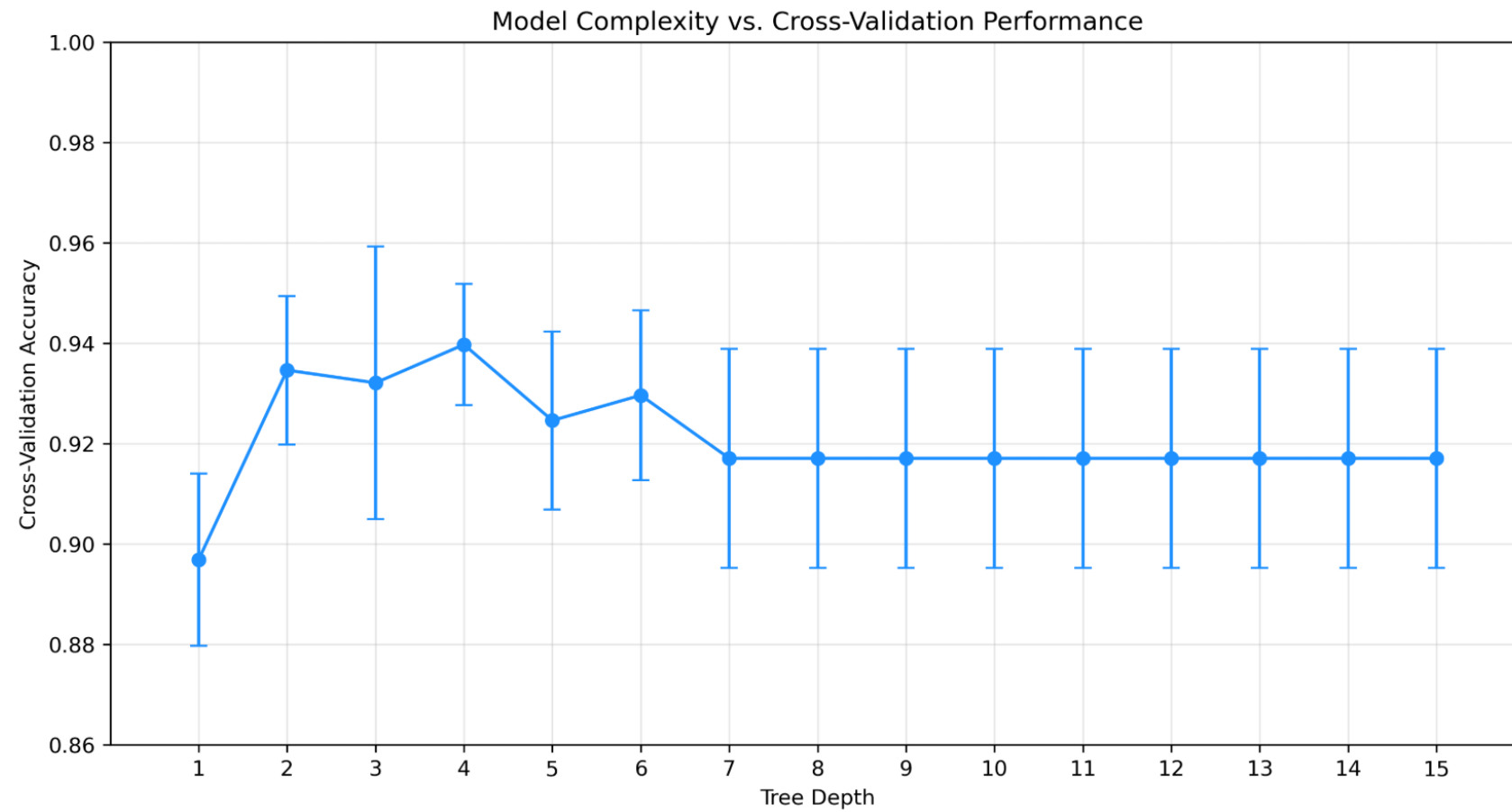
Lab Practice

```
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score
cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
mean_cv = []
std_cv = []
for depth in depths:
    model = DecisionTreeClassifier(max_depth=depth,random_state=42)
    scores=cross_val_score(model,X_train,y_train,cv=cv,scoring="accuracy")
    mean_cv.append(scores.mean())
    std_cv.append(scores.std())
```

Lab Practice

```
plt.figure(figsize=(12, 6),dpi=300)
plt.errorbar(depths,mean_cv,yerr=std_cv,marker='o',
             c='dodgerblue',capsize=4,linestyle='-')
plt.xlabel("Tree Depth")
plt.ylabel("Cross-Validation Accuracy")
plt.title("Model Complexity vs. Cross-Validation Performance")
plt.xticks(depths)
plt.axis([0,16,0.86,1])
plt.grid(alpha=0.3)
plt.show()
```

Lab Practice



Lab Practice

mean_cv

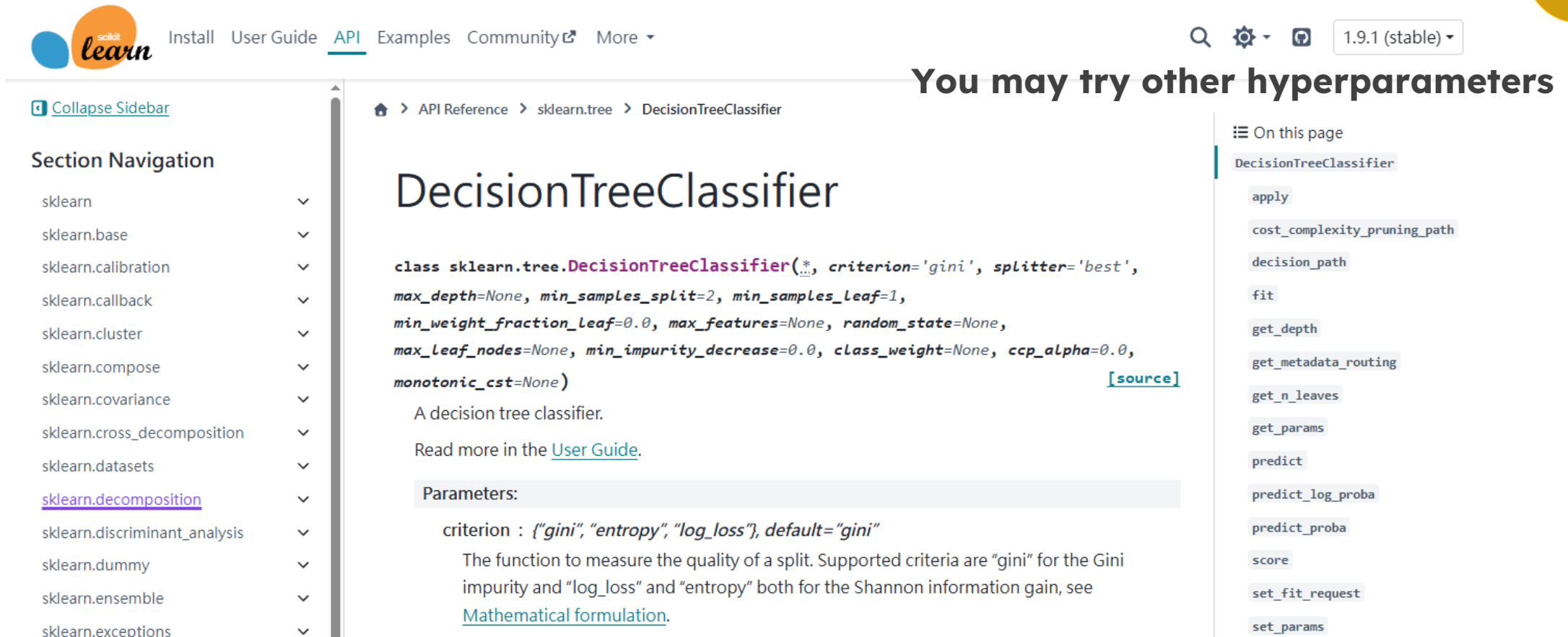
```
[np.float64(0.8968987341772152),  
 np.float64(0.9346518987341772),  
 np.float64(0.9321202531645569),  
 np.float64(0.9397468354430378),  
 np.float64(0.9246202531645571),  
 np.float64(0.9296518987341772),  
 np.float64(0.9170886075949367),  
 np.float64(0.9170886075949367),  
 np.float64(0.9170886075949367),  
 np.float64(0.9170886075949367),  
 np.float64(0.9170886075949367),  
 np.float64(0.9170886075949367),  
 np.float64(0.9170886075949367),  
 np.float64(0.9170886075949367),  
 np.float64(0.9170886075949367),  
 np.float64(0.9170886075949367)]
```

best_depth = **list**(depths)[**np.argmax**(mean_cv)]
best_depth

4

Lab Practice

<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>



The screenshot shows the scikit-learn website's API reference for `DecisionTreeClassifier`. The page includes a sidebar with navigation links, a main content area with the class definition and description, and a right sidebar listing methods. A large black box redacts the 'Parameters' section of the main content.

Section Navigation

- sklearn
- sklearn.base
- sklearn.calibration
- sklearn.callback
- sklearn.cluster
- sklearn.compose
- sklearn.covariance
- sklearn.cross_decomposition
- sklearn.datasets
- sklearn.decomposition
- sklearn.discriminant_analysis
- sklearn.dummy
- sklearn.ensemble
- sklearn.exceptions

DecisionTreeClassifier

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini', splitter='best',
max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features=None, random_state=None,
max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None, ccp_alpha=0.0,
monotonic_cst=None)
```

A decision tree classifier.

Read more in the [User Guide](#).

Parameters:

`criterion` : {"gini", "entropy", "log_loss"}, default="gini"

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see [Mathematical formulation](#).

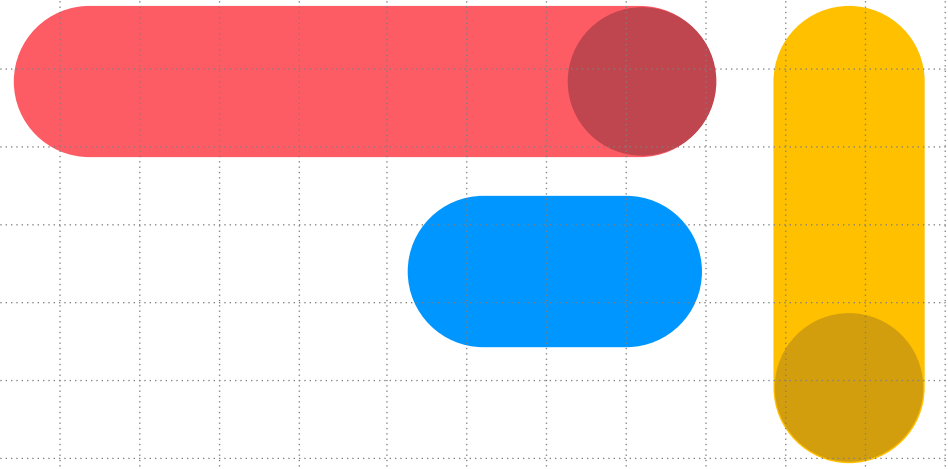
On this page

- DecisionTreeClassifier
- apply
- cost_complexity_pruning_path
- decision_path
- fit
- get_depth
- get_metadata_routing
- get_n_leaves
- get_params
- predict
- predict_log_proba
- predict_proba
- score
- set_fit_request
- set_params

References

If you want to learn more in details, please see the following references.

- 1) Google Machine Learning :: Foundational courses
- 2) IBM Machine Learning
- 3) Abu-Mostafa, Y. S., Magdon-Ismail, M., & Lin, H. T. (2012). Learning from data (Vol. 4, p. 4). New York: AMLBook.



The End

Thank you for your attention!

Email: chchan@ntnu.edu.tw

Website: <https://toodou.github.io/>